QUANGVY ORCHESTRATION

agent-brain

Hướng dẫn sử dụng — CLI và Dashboard web

Một file binary duy nhất để giao việc cho AI agent theo giao thức Lead → `.plan.md` → Dev → QA: đúng dự án, đúng vai trò, kill sạch cây tiến trình, đo được token. Tài liệu này dành cho người dùng hằng ngày và cho việc review, báo lỗi, đề xuất nâng cấp.



Phiên bản phần mềm	**agent-brain 0.1.0** (binary tĩnh musl, Linux x86_64)
Phạm vi	Phase 1–3 (giao thức, lõi CLI) và Phase 5 (Dashboard nhúng)
Ngày	19/10/2026
Mã nguồn	github.com/vinhhx/quang-vy-ai-orchestration

Mục lục

1. Tổng quan: agent-brain làm gì
2. Cài đặt và kiểm tra
3. Chuẩn bị: dự án, profile, cấu hình
4. Quy trình chuẩn bằng CLI: Lead → Dev → QA
5. Lệnh nhanh, dừng tiến trình và thống kê
6. Dashboard web: khởi động và đăng nhập
7. Dashboard: năm màn hình
8. Dữ liệu, file và an toàn
9. Xử lý sự cố
10. Hạn chế đã biết (trước khi báo lỗi)
11. Review, báo lỗi và đề xuất nâng cấp
12. Phụ lục: mã thoát, khóa cấu hình, HTTP API

> **Quy ước trong tài liệu**
>
> `PROJECT` là tên, id hoặc alias dự án (gõ tiếng Việt có dấu hoặc không dấu đều được). `~` là thư mục home của bạn. Lệnh in trong khung tối là lệnh gõ ở terminal.

1. Tổng quan: agent-brain làm gì

agent-brain là bộ điều phối nhẹ, chạy bằng **một file binary**, dùng để giao việc cho AI agent (hiện tại là Claude Code CLI) trên các dự án của bạn. Nó thay thế bộ script Python `brain` cũ và dùng chung danh mục dự án với bộ script đó.

Giao thức ba vai trò

1. Lead	2. Bạn duyệt plan	3. Dev	4. QA
Đọc yêu cầu, tạo nhánh `task/<slug>`, viết `.plan.md` và commit `<slug>: plan`. Chỉ được ghi `.plan.md`.	Đọc, sửa nếu cần bằng editor của bạn, commit lại. Không có bước nào tự chạy tiếp.	Sửa đúng các file trong bảng *Thay đổi* của plan. Sửa file ngoài danh sách là vi phạm giao thức (mã thoát 3).	Kiểm tra `git diff` từ commit plan đến HEAD, ghi `.qa-report.md` : *Đạt* hoặc *Cần sửa*. Không sửa code.

Sau khi QA **Đạt**, bạn tự review và merge nhánh `task/<slug>`. agent-brain không tự merge.

Những đảm bảo chính

- **Đúng dự án (Fallback Ask):** tên dự án khớp 0 hoặc nhiều hơn 1 dự án thì dừng và in danh sách để bạn chọn (mã thoát 2). Không bao giờ đoán.
- **Kill sạch:** mỗi lần chạy agent nằm trong process group riêng; `kill` dừng cả cây tiến trình con, kiểm tra PID chưa bị tái sử dụng trước khi gửi tín hiệu.
- **Đo được token:** mọi sự kiện stream-json được lưu vào SQLite; `stats` và màn hình Thống kê tổng hợp token và chi phí tương đương.
- **Hai tài khoản tách biệt:** profile `work` (`~/.claude`) và `personal` (`~/.claude-orca`), gán theo dự án.

Hai cách dùng

Cách	Khi nào dùng
CLI (`agent-brain run …`)	Làm việc trong terminal, script hóa, chạy trên máy không có trình duyệt.
Dashboard (`agent-brain serve`)	Xem task và log trực tiếp, giao việc, bấm Kill, xem thống kê, sửa alias dự án. Chạy ngay trong binary, chỉ trên `127.0.0.1`.

Hai cách dùng chung một cơ sở dữ liệu: run khởi chạy ở terminal hiện lên dashboard ngay, và có thể Kill từ dashboard.

2. Cài đặt và kiểm tra

Yêu cầu

- Linux x86_64. Binary là file tĩnh, không cần thư viện hay runtime nào khác.
- **Claude Code CLI** (`claude`) đã đăng nhập cho profile bạn dùng. Cho profile `personal` : đăng nhập với `CLAUDE_CONFIG_DIR=~/.claude-orca` .
- **git** trong `PATH` .
- Trình duyệt bất kỳ (chỉ cần cho dashboard).

Cài binary

```
# Build từ mã nguồn (offline, khóa phiên bản)
bash scripts/build-release.sh

# Cài vào ~/.local/bin (thư mục này phải có trong PATH)
install -m 755 agent-brain/target/x86_64-unknown-linux-musl/release/agent-brain
~/.local/bin/agent-brain

# Kiểm tra
agent-brain --version       # agent-brain 0.1.0
agent-brain --help
```

Nếu máy không có `musl-gcc` , script build tự chuyển sang `gcc` và in cảnh báo; kết quả vẫn là binary tĩnh.

Danh sách lệnh

Lệnh	Việc làm
`projects scan`	Quét thư mục gốc (độ sâu 1) tìm repo git, cập nhật danh mục. Giữ nguyên phần bạn đã sửa tay.
`projects add`	Thêm thủ công một dự án (`--type code|media`).
`run lead|dev|qa`	Chạy một vai trò của giao thức.
`status`	Trạng thái git và run gần nhất. 0 token.
`inspect`	Nhờ Claude đọc nhanh diff chưa commit (1 lượt, không lưu session).
`kill`	Dừng task đang chạy và cả cây tiến trình con.
`stats`	Thống kê token và chi phí tương đương.
`serve`	Mở dashboard web.
`guard`	Hook nội bộ do Claude CLI gọi để chặn ghi file ngoài plan. Bạn không cần gọi trực tiếp.

3. Chuẩn bị: dự án, profile, cấu hình

3.1 Danh mục dự án

File: `~/.config/agent-brain/projects_map.json`. Cách nhanh nhất là quét thư mục chứa các repo:

```
agent-brain projects scan --root ~/code --root ~/work
agent-brain projects add ~/code/kenh-video --type media
```

Mỗi dự án có dạng:

```
{
  "id": "web-ban-hang",
  "path": "/duong/dan/toi/repo",
  "name_vi": "Web bán hàng",
  "aliases": ["shop"],
  "type": "code",
  "repo": null,
  "profile": "work",
  "missing": false
}
```

Trường	Ý nghĩa
`id`	Định danh duy nhất (slug).
`name_vi`, `aliases`	Tên và biệt danh để gọi dự án. Có thể gõ có dấu hoặc không dấu. Sửa alias được trên dashboard (màn hình Dự án).
`profile`	**Bắt buộc**: `work` hoặc `personal`. Dự án có `profile: ""` sẽ bị từ chối chạy agent.
`missing`	`true` khi thư mục không còn trên đĩa; dự án này bị bỏ qua khi chọn.

3.2 Profile tài khoản

Profile	Thư mục cấu hình Claude	Ghi chú
`work`	`~/.claude` (mặc định)	Tài khoản công việc.
`personal`	`~/.claude-orca`	**Tối đa 1 run cùng lúc** trên toàn hệ thống (quy tắc E-06), vì thư mục này có thể dùng chung với phiên Claude bạn đang mở.

Khi chạy agent, agent-brain bỏ đặt `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, `ANTHROPIC_BASE_URL` để agent luôn dùng đúng tài khoản đã đăng nhập của profile.

> **Không ghi đè profile khi giao việc**
> Profile luôn lấy từ dự án. Muốn đổi tài khoản cho một dự án, sửa trường `profile` trong `projects_map.json` (dashboard không cho sửa trường này).

3.3 File cấu hình (tùy chọn)

File `~/.config/agent-brain/config.json`. Không có file thì dùng mặc định nhúng sẵn. Các khóa hay chỉnh:

Khóa	Mặc định	Ý nghĩa
`models`	lead: opus, dev/qa: sonnet	Model theo vai trò.
`timeout_seconds`	lead/dev 1800, qa 1200, inspect 300	Thời gian chạy tối đa (giây); quá hạn thì run kết thúc `timed_out`.
`kill_grace_secs`	3	Chờ giữa SIGINT và SIGKILL khi dừng.
`dashboard_port`	7437	Cổng dashboard.
`claude_bin`	`claude`	Đường dẫn Claude CLI.

Biến môi trường: `AGENT_BRAIN_CONFIG` (file cấu hình khác), `AGENT_BRAIN_DB` (database khác, tiện để thử nghiệm cách ly). Danh sách đầy đủ ở Phụ lục.

4. Quy trình chuẩn bằng CLI: Lead → Dev → QA

Bước 1 — Lead viết plan

```
agent-brain run lead "web bán hàng" "Thêm nút xuất đơn hàng ra CSV ở trang quản trị"
# tùy chọn: --name xuat-csv   (đặt slug cho task)
#           --dry-run         (chạy thử, không sinh tác vụ)
#           --verbose         (in chi tiết)
```

Kết quả: nhánh `task/<slug>`, file `.plan.md` ở gốc repo và commit `<slug>: plan`. Terminal in cặp **task N, run M** — ghi lại số task để dùng khi kill hoặc báo lỗi.

Nếu tên dự án mơ hồ, lệnh dừng với mã 2 và in danh sách dự án khớp. Gõ lại với id chính xác.

Bước 2 — Bạn duyệt plan

- Mở `.plan.md` trên nhánh task. Kiểm tra bảng **Thay đổi**: Dev chỉ được sửa các file liệt kê ở đây.
- Nếu cần sửa plan: sửa bằng editor của bạn rồi **commit**. Plan có thay đổi chưa commit sẽ chặn bước Dev trên dashboard.

Bước 3 — Dev thực hiện

```
agent-brain run dev "web bán hàng"
```

Dev làm việc trên nhánh task hiện tại và chỉ được ghi các file trong plan (hook `guard` chặn ghi ngoài danh sách). Có hai kết thúc bất thường:

- **Vướng mắc** (mã thoát 4): Dev thiếu thông tin, ghi mục *Vướng mắc* vào `.plan.md` và dừng an toàn. Bạn bổ sung plan, commit, rồi chạy lại Dev (hoặc chạy lại Lead).
- **Vi phạm giao thức** (mã thoát 3): có file ngoài plan bị sửa. Xem lại diff trước khi làm tiếp.

Bước 4 — QA kiểm tra

```
agent-brain run qa "web bán hàng"
```

QA đọc `git diff` từ commit plan đến HEAD (bỏ qua `.plan.md`, `.qa-report.md`) và ghi `.qa-report.md`:

- **Đạt** → bạn review và merge nhánh.
- **Cần sửa** → chạy lại Dev theo báo cáo, hoặc chạy lại Lead để làm lại plan.

> **Mẹo**
> Trước khi chạy QA, dùng `agent-brain status PROJECT` (0 token) để xem nhánh và diffstat, hoặc `agent-brain inspect PROJECT` để nhờ Claude đọc nhanh diff chưa commit.

Quy tắc chạy song song

- Mỗi dự án chỉ có **một run** tại một thời điểm.
- Profile `personal` chỉ có **một run** trên toàn hệ thống.
- Các dự án `work` khác nhau có thể chạy song song.

5. Lệnh nhanh, dừng tiến trình và thống kê

status — 0 token

```
agent-brain status "web bán hàng"
```

Không gọi Claude. Hiển thị trạng thái git, nhánh hiện tại, diffstat và run gần nhất trong database.

inspect — đọc nhanh diff

```
agent-brain inspect "web bán hàng" [--verbose]
```

Gọi Claude đúng 1 lượt, không công cụ, không lưu session. Diff lớn hơn `inspect_max_diff_bytes` (100.000 byte) được cắt gọn và có thông báo.

kill — dừng task

```
agent-brain kill 12        # 12 là TASK_ID
```

1. Đọc PID và thời điểm khởi tạo của run từ database, đối chiếu với `/proc`. Nếu PID đã bị hệ điều hành tái sử dụng, **không** gửi tín hiệu.
2. Gửi SIGINT tới cả process group, chờ `kill_grace_secs`, rồi SIGKILL những tiến trình còn sót.
3. Run được ghi `killed`; terminal đang chạy run đó thoát với mã 130.

Ở terminal đang chạy: **Ctrl+C một lần** = dừng mềm; **Ctrl+C hai lần nhanh** = dừng ngay (dưới 1 giây). Đóng cửa sổ terminal cũng dừng cả cây tiến trình.

Trạng thái kết thúc bất thường của run: `killed` (bị dừng có chủ đích), `timed_out` (quá thời gian), `lost` (tiến trình biến mất bất thường).

stats — token và chi phí

```
agent-brain stats
agent-brain stats --since 7d --profile work
agent-brain stats --since 24h --json      # JSON: total, by_project, by_role
```

`--since` nhận `Nm`, `Nh`, `Nd` (phút, giờ, ngày). Chi phí là **ước tính tương đương USD** theo bảng giá chuẩn của từng model (input, output, cache read, cache creation) — không phải hóa đơn thật.

6. Dashboard web: khởi động và đăng nhập

Khởi động

```
agent-brain serve              # cổng 7437
agent-brain serve --port 8123  # cổng khác; --port 0 = hệ điều hành tự chọn
```

Terminal in một URL có token, dạng:

```
agent-brain dashboard: http://127.0.0.1:7437/?token=<64 ký tự hex>
```

1. Mở **đúng URL đó** trong trình duyệt (dùng `127.0.0.1`, không dùng `localhost`).
2. Server đổi token thành cookie phiên và chuyển bạn về `/`; thanh địa chỉ không còn token.
3. Giữ terminal mở. **Ctrl+C** dừng dashboard — các run do dashboard khởi chạy cũng dừng theo.

Đặc điểm cần biết

- **Token mới mỗi lần khởi động**, không ghi xuống đĩa. Khởi động lại `serve` thì cookie cũ hết hiệu lực — mở URL mới.
- **Chỉ một** `serve` tại một thời điểm. Chạy cái thứ hai sẽ bị từ chối kèm PID và cổng của cái đang chạy.
- **Chỉ nghe** `127.0.0.1`: máy khác trong mạng LAN không vào được. Đây là chủ ý vì dashboard chạy được lệnh trên máy bạn.
- **Chạy offline hoàn toàn**: CSS, JavaScript, font đều nằm trong binary.

> **Không chia sẻ URL có token**
> Ai có token đều điều khiển được agent trên các dự án của bạn. Không dán URL vào chat, issue, log hay ảnh chụp màn hình.

Bố cục chung

- **Thanh trên:** 5 mục điều hướng (Tổng quan, Giao việc, Thống kê, Dự án; Chi tiết task mở từ thẻ task), bộ lọc profile (**Tất cả / work / personal**) và nút đổi giao diện sáng/tối.
- **Bộ lọc profile chỉ lọc những gì bạn thấy** (task, thống kê, gợi ý dự án). Nó không đổi tài khoản chạy task.
- Dữ liệu tự cập nhật trực tiếp. Mất kết nối thì hiện thông báo *"Không cập nhật được — đang thử lại..."* và tự kết nối lại.
- Dùng được bằng bàn phím: Tab giữa các điều khiển, Escape đóng hộp thoại.

7. Dashboard: năm màn hình

7.1 Tổng quan `#/`

Task chia thành 4 nhóm: **Đang chạy**, **Chờ bạn** (ví dụ chờ duyệt plan), **Lỗi / cần xử lý** (Vướng mắc, Cần sửa, killed, timed_out…), **Xong**. Bấm thẻ task để mở Chi tiết. Dùng *Hiện thêm 10* để xem task cũ hơn.

7.2 Giao việc `#/assign`

1. **Dự án:** gõ tên, alias hoặc tên không dấu rồi **chọn trong danh sách gợi ý** (chuột, hoặc ↑/↓ rồi Enter). Dự án *thiếu thư mục* hiện mờ và không chọn được.
2. **Yêu cầu (tiếng Việt):** mô tả việc cần làm.
3. **Tên task (slug):** tự tạo từ yêu cầu giống CLI; sửa được.
4. Kiểm tra khung thông tin (dự án, nhánh hiện tại, profile) rồi bấm **Giao việc** để chạy Lead.

Nếu tên khớp nhiều dự án, màn hình hiện *"Tên khớp nhiều dự án — chọn một"* và nút Giao việc bị khóa cho tới khi bạn chọn. Nút bị khóa luôn kèm lý do (dự án đang bận, đã có run personal…).

7.3 Chi tiết task `#/task/N`

- **Stepper Lead → Dev → QA** cho biết bước hiện tại và kết quả từng bước.
- **Plan** và **báo cáo QA** hiển thị chỉ đọc. Sửa plan bằng editor của bạn rồi commit.
- **Nút hành động theo ngữ cảnh:** *Chạy Dev* (plan sẵn sàng và đã commit sạch), *Chạy QA* (Dev xong), *Chạy lại Lead* (Vướng mắc hoặc Cần sửa). Mỗi bước chỉ chạy khi bạn bấm — không có gì tự chạy tiếp.
- **Kill run:** hiện khi có run đang chạy. Hộp xác nhận mặc định chọn nút an toàn (*Giữ chạy tiếp*); *Chạy lại Lead* cũng có xác nhận (*Giữ plan hiện tại*).
- **Log trực tiếp:** tự cuộn theo dòng mới. Cuộn lên để tạm dừng, bấm *Về cuối (live)* để theo lại. Bấm một dòng log để xem JSON thô của sự kiện. Tải lại trang giữa chừng vẫn giữ log cũ và nhận tiếp log mới.
- Run bị kill hiển thị chi phí `–` (không đủ dữ liệu để tính), kèm số token đã ghi được.

7.4 Thống kê `#/stats`

Biểu đồ token và chi phí tương đương theo ngày, theo vai trò (lead, dev, qa, inspect), theo dự án và theo profile. Cùng số liệu với `agent-brain stats`; theo bộ lọc profile ở thanh trên.

7.5 Dự án `#/projects`

- Danh sách dự án trong `projects_map.json`, dự án mất thư mục được đánh dấu.
- **Sửa alias** rồi bấm *Lưu alias*. Alias trùng với dự án khác bị từ chối. Nếu danh mục vừa bị sửa ở nơi khác trong lúc bạn đang sửa, thay đổi của bạn không ghi đè: dashboard báo lỗi, tải lại dữ liệu mới, và bạn nhập lại alias.
- **Quét lại** = `agent-brain projects scan`.
- Không sửa được profile, id, đường dẫn hay loại dự án trên web (sửa trong file).

8. Dữ liệu, file và an toàn

Vị trí file

Đường dẫn	Nội dung
`~/.config/agent-brain/projects_map.json`	Danh mục dự án.
`~/.config/agent-brain/config.json`	Cấu hình (tùy chọn).
`~/.local/share/agent-brain/brain.db`	SQLite: task, run, sự kiện, token. Quyền `0600`.
`~/.local/share/agent-brain/runs/`	Log chi tiết từng run. Quyền thư mục `0700`.
`serve.lock` (thư mục dữ liệu)	Khóa một-dashboard: PID, cổng, giờ khởi động. Không chứa token.
`.plan.md`, `.qa-report.md` (gốc repo)	Plan của Lead và báo cáo của QA, trên nhánh task.

Mọi dòng stream-json thô từ Claude được lưu nguyên vẹn (cột `raw_json` của bảng `events`) để kiểm tra lại bất cứ lúc nào.

Nguyên tắc an toàn

- Token dashboard tương đương quyền chạy lệnh trên máy bạn — giữ kín (xem mục 6).
- **Rủi ro đã chấp nhận:** cookie không tách theo cổng (RFC 6265). Một dịch vụ web khác đang chạy trên `127.0.0.1` về lý thuyết có thể nhận được cookie dashboard. Khi không dùng dashboard, hãy tắt `serve`; khi đang chạy, tránh mở trang của dịch vụ cục bộ lạ trên cùng trình duyệt.
- Agent không được gán API key: các biến `ANTHROPIC_*` bị bỏ đặt, agent dùng tài khoản đăng nhập của profile.
- Không dừng tiến trình agent bằng `pkill` / `killall`; dùng `agent-brain kill` hoặc nút Kill để dọn sạch cây tiến trình và cập nhật database.

9. Xử lý sự cố

Hiện tượng	Cách xử lý
Dashboard báo **401** / phiên hết hạn	Mở lại URL đầy đủ có `?token=` đang in ở terminal `serve`. Sau khi khởi động lại serve, token cũ không dùng được nữa.
421 hoặc không đăng nhập được qua `localhost`	Dùng đúng `http://127.0.0.1:PORT/…`.
`serve` báo đã có dashboard khác chạy	Dùng dashboard đó (PID và cổng được in ra), hoặc dừng nó bằng Ctrl+C ở terminal của nó.
Cổng bị chiếm (*Address in use*)	`--port <cổng>`, `--port 0`, hoặc khóa `dashboard_port`.
Máy khác trong LAN không vào được	Đúng thiết kế: dashboard chỉ nghe `127.0.0.1`.
Mã thoát **2** khi chạy lệnh	Tên dự án mơ hồ — dùng id trong danh sách được in ra.
Mã thoát **1**, "Không tìm thấy .plan.md"	Chạy Lead trước, hoặc chuyển sang đúng nhánh `task/<slug>`.
Dự án bị từ chối vì profile rỗng	Đặt `"profile": "work"` hoặc `"personal"` trong `projects_map.json`.
Nút Chạy Dev/QA bị khóa	Đọc lý do hiện cạnh nút: dự án đang bận, đã có run personal, plan chưa commit...
Run treo lâu	Bấm Kill (hoặc `agent-brain kill TASK_ID`). Run tự dừng khi quá `timeout_seconds`.
Dữ liệu dashboard không cập nhật	Thấy thông báo "đang thử lại" thì chờ; vẫn không được thì tải lại trang (F5).

10. Hạn chế đã biết (trước khi báo lỗi)

Các mục dưới đây đã được ghi nhận ở lần review Phase 5 (file `.planning/phases/05-embedded-dashboard/05-REVIEW.md` và `05-UI-REVIEW.md`) và đang chờ sửa. Nếu bạn gặp đúng hiện tượng này, chỉ cần ghi thêm bằng chứng/ảnh hưởng thực tế, không cần báo mới.

Mã	Hiện tượng bạn có thể thấy	Ghi chú
WR-03	Đóng hộp thoại (Kill, Chạy lại Lead, JSON thô) xong, focus bàn phím không quay về nút đã mở nó.	Ảnh hưởng người dùng bàn phím.
WR-04	Giao việc: gõ tên rồi nhấn Enter quá nhanh (< 0,2 giây) có thể chọn từ danh sách gợi ý cũ.	Luôn kiểm tra dự án trong khung thông tin trước khi bấm Giao việc.
WR-05	Hai lần giao việc cho hai dự án `personal` gần như cùng lúc có thể cùng được nhận.	Lớp chạy `run` kiểm tra lại; vẫn nên tránh.
WR-06	Hai dự án có nhánh task trùng tên có thể làm sai trạng thái "đang chạy".	Hiếm.
WR-11	Kill đúng lúc run đổi bước có thể dừng run mới của cùng task.	Cửa sổ vài mili giây.
WR-09/10	Chuyển màn hình hoặc mở JSON thô liên tục quá nhanh: phản hồi cũ có thể ghi đè màn hình mới.	Tải lại trang là hết.
WR-01	Giá trị `--since` cực lớn có thể cho kết quả sai.	Dùng giá trị hợp lý (ví dụ `90d`).
WR-13	Lỗi 500 của API có thể hiện thông báo kỹ thuật kèm đường dẫn nội bộ.	Xóa đường dẫn riêng tư trước khi dán vào issue.
WR-02	Mục "Danh mục HTTP API" trong `agent-brain/README.md` liệt kê sai vài route.	Danh sách đúng ở Phụ lục C.
UI	Mũi tên `→` giữa các bước trong stepper hơi mờ ở giao diện sáng.	Thẩm mỹ.
UI-04	Nhãn ngày trên biểu đồ Thống kê chưa được kiểm với dữ liệu > 14 ngày.	Báo nếu thấy chữ chồng nhau.

Chưa có: chạy engine khác ngoài Claude (Phase 6), tối ưu token bằng tóm tắt/handover (Phase 4), chỉnh plan trên web (chủ ý không làm).

11. Review, báo lỗi và đề xuất nâng cấp

11.1 Gợi ý review theo từng phần

Phần	Những điều nên thử
Giao việc & định tuyến	Tên có dấu/không dấu, alias, tên khớp nhiều dự án, dự án mất thư mục.
Chất lượng plan	Plan có đủ bảng Thay đổi? Dev có làm đúng phạm vi? Có lần nào Vướng mắc vô lý?
QA	Báo cáo có bắt được lỗi thật? Có lần "Đạt" sai?
Kill & độ bền	Kill giữa chừng, Ctrl+C, đóng terminal, tắt dashboard khi đang chạy; sau đó kiểm tra không còn tiến trình `claude` sót.
Dashboard	Log trực tiếp, tải lại trang giữa chừng, giao diện sáng/tối, màn hình hẹp (điện thoại, 480–768 px), dùng toàn bàn phím.
Token & chi phí	Số liệu Thống kê khớp với cảm nhận? Vai trò nào tốn nhất? (Dữ liệu này là đầu vào cho Phase 4.)

11.2 Thu thập thông tin trước khi báo

```
agent-brain --version
agent-brain status PROJECT                # trạng thái repo, run gần nhất
agent-brain stats --since 24h --json      # nếu liên quan token/chi phí
claude --version                          # phiên bản Claude CLI
```

- **Số task và số run** (in ở terminal, hoặc trên URL `#/task/N` của dashboard).
- Mã thoát của lệnh: `echo $?` ngay sau khi lệnh kết thúc.
- Log của run trong `~/.local/share/agent-brain/runs/` và phần stderr của terminal `serve` nếu lỗi ở dashboard.
- Ảnh chụp màn hình (đã che token) và kích thước cửa sổ trình duyệt nếu là lỗi giao diện.

> **Trước khi gửi: xóa thông tin nhạy cảm**
> Không đưa token dashboard (`?token=…`), cookie, API key, nội dung mã nguồn riêng của khách hàng hay đường dẫn home đầy đủ vào issue.

11.3 Mẫu báo lỗi

```
Tiêu đề: [dashboard|cli|plan|qa|kill|stats] Mô tả ngắn

Phiên bản: agent-brain 0.1.0, claude x.y.z, trình duyệt …
Profile: work | personal      Task/Run: task 12, run 31
Mức độ: chặn | nặng | nhẹ | thẩm mỹ

Các bước tái hiện:
1. …
2. …

Kết quả mong đợi: …
Kết quả thực tế: …  (mã thoát, thông báo lỗi)
Bằng chứng: log/ảnh (đã che token)
Liên quan hạn chế đã biết? (WR-xx / không)
```

11.4 Mẫu đề xuất nâng cấp

```
Tiêu đề: [đề xuất] …
Vấn đề hiện tại (đau ở đâu, bao lâu gặp một lần): …
Đề xuất: …
Lợi ích đo được (thời gian, token, số lần thao tác): …
Phạm vi: CLI | dashboard | giao thức | cấu hình
Ưu tiên của bạn: cao | vừa | thấp
```

11.5 Mức độ

Mức	Khi nào
Chặn	Không chạy được Lead/Dev/QA, chạy nhầm dự án, mất dữ liệu, tiến trình sót sau Kill.
Nặng	Sai trạng thái, sai số liệu, chức năng không làm đúng nhưng có cách vòng.
Nhẹ	Chậm, khó dùng, thông báo khó hiểu.
Thẩm mỹ	Màu, chữ, khoảng cách, căn chỉnh.

11.6 Gửi ở đâu

Tạo issue trên kho mã **github.com/vinhhx/quang-vy-ai-orchestration** (tab Issues), hoặc gửi trực tiếp cho người phụ trách dự án. Mỗi lỗi một issue giúp lập kế hoạch sửa (fix plan) dễ hơn.

12. Phụ lục

A. Mã thoát

Mã	Tên	Ý nghĩa
0	EXIT_OK	Thành công (kể cả QA kết luận Đạt hoặc Cần sửa hợp lệ).
1	EXIT_ERROR	Lỗi thực thi, cấu hình sai, thiếu file, điều kiện git không thỏa.
2	EXIT_ROUTING	Tên dự án khớp 0 hoặc nhiều hơn 1 dự án (Fallback Ask).
3	EXIT_PROTOCOL	Vi phạm giao thức: sửa file ngoài danh sách của plan.
4	EXIT_BLOCKED	Dev gặp Vướng mắc, đã ghi vào `.plan.md` và dừng an toàn.
130	EXIT_INTERRUPTED	Dừng bởi Ctrl+C hoặc `agent-brain kill`.

B. Khóa cấu hình đầy đủ

Khóa	Mặc định	Ý nghĩa
`claude_bin`	`"claude"`	Lệnh Claude CLI.
`models`	lead opus, dev sonnet, qa sonnet	Model theo vai trò.
`inspect_model`	`"sonnet"`	Model cho `inspect`.
`profiles`	work, personal	`claude_config_dir` và các biến môi trường bị bỏ đặt cho từng profile.
`common_args`	`["--strict-mcp-config"]`	Đối số chung cho Claude CLI.
`max_turns`	null (không giới hạn)	Giới hạn lượt theo vai trò.
`timeout_seconds`	1800 / 1800 / 1200 / 300	lead / dev / qa / inspect.
`runs_dir`	null → `~/.local/share/agent-brain/runs/`	Thư mục log run.
`kill_grace_secs`	3	Chờ giữa SIGINT và SIGKILL.
`inspect_max_diff_bytes`	100000	Giới hạn diff gửi cho `inspect`.
`dashboard_port`	7437	Ưu tiên: `--port` > khóa này > 7437.

C. HTTP API của dashboard

Mọi route cần cookie phiên (trừ `GET /?token=…` dùng để lấy cookie). Lệnh ghi (POST/PUT) còn kiểm tra Origin và Content-Type. Dành cho người phát triển; người dùng thường không cần gọi trực tiếp.

Route	Việc làm
`GET /api/tasks`	Danh sách task, trạng thái bận của dự án (lọc `?profile=`).
`POST /api/tasks`	Tạo task và chạy Lead. Body: `project`, `request`, `name`.
`GET /api/tasks/{id}`	Chi tiết task: các bước, plan, báo cáo QA, nút được phép.
`POST /api/tasks/{id}/dev|qa|relead`	Chạy Dev, QA, hoặc chạy lại Lead.

`GET /api/preflight`	Kiểm tra trước khi giao việc (dự án, nhánh, profile, lý do khóa).
`GET /api/jobs/{id}`	Trạng thái job do dashboard khởi chạy.
`GET /api/runs/{id}`	Thông tin và sự kiện của run.
`GET /api/runs/{id}/stream`	Log trực tiếp (Server-Sent Events).
`GET /api/runs/{id}/events/{seq}/raw`	JSON thô của một sự kiện.
`POST /api/runs/{id}/kill`	Dừng run (qua cùng cơ chế với `agent-brain kill`).
`GET /api/projects`	Danh mục dự án kèm `etag`.
`PUT /api/projects/{id}`	Sửa alias. Body: `aliases`, `etag`.
`POST /api/projects/scan`	Quét lại thư mục dự án.
`GET /api/projects/suggest?q=…`	Gợi ý dự án theo độ khớp tên/alias.
`GET /api/projects/{id}/status`	Trạng thái git nhanh (giống `status`).
`GET /api/stats?profile=…&since=…`	Thống kê token và chi phí.
`GET /api/slug?text=…`	Tạo slug từ văn bản tiếng Việt.